Proxmox VE Introduction

Day 2: Virtual Machines & Containers

Day 2 — Agenda

1. Recap: Day 1 Key Points
2. QEMU/KVM Fundamentals
3. Creating & Configuring Virtual Machines
4. VM Templates & Cloning
5. LXC Containers — Concepts & Use Cases
6. Containers vs. VMs — Choosing the Right Workload
7. Resource Management — CPU, Memory, Disk
8. Snapshots & Live Migration
9. VirtIO Drivers & the QEMU Guest Agent
10. Hands-on Lab: Deploying VMs and Containers

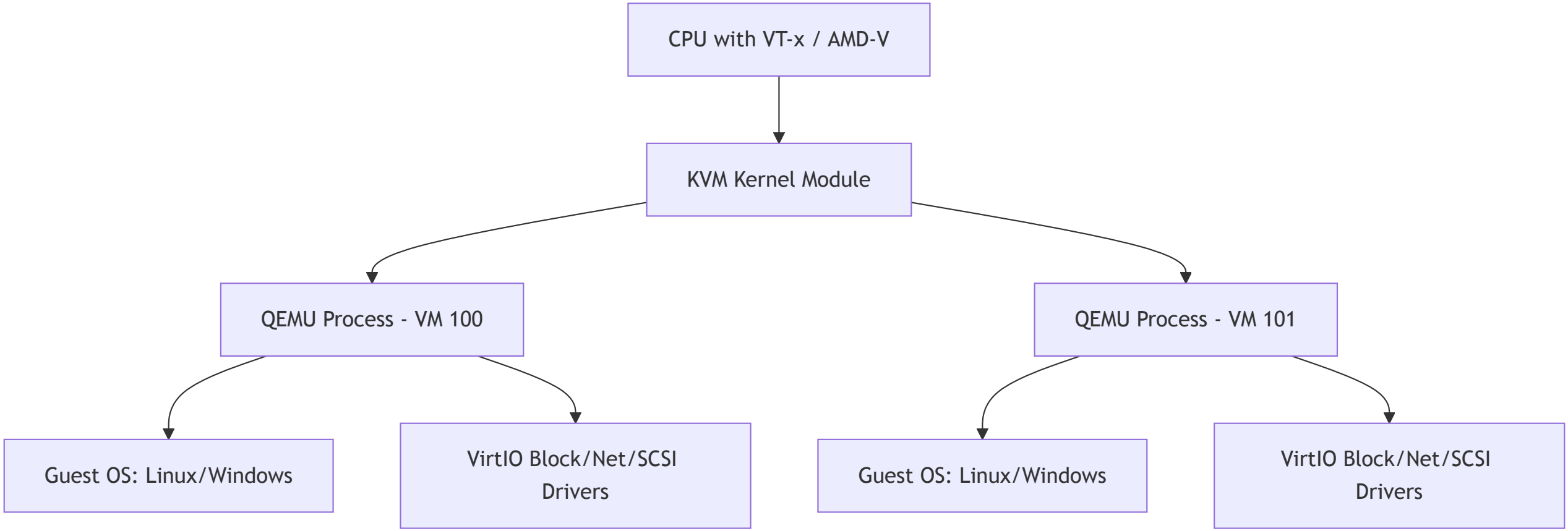
1. Recap: Day 1 Key Points

- Proxmox VE = **Type 1 hypervisor** on Debian, built on **KVM** + **LXC**
- `/etc/pve` (**pmxcfs**) replicates configuration across the whole cluster
- Users/Realms/Roles/ACL paths control **who can do what**
- Storage and networking backends are pluggable per workload

Today we go one layer down: how **VMs** and **containers** actually run.

2. QEMU/KVM Fundamentals

KVM (Kernel-based Virtual Machine) is a Linux kernel module that exposes hardware virtualization extensions (Intel VT-x / AMD-V) to user space. **QEMU** is the user-space process that uses KVM to emulate a full virtual machine — CPU, chipset, disk controllers, NICs.

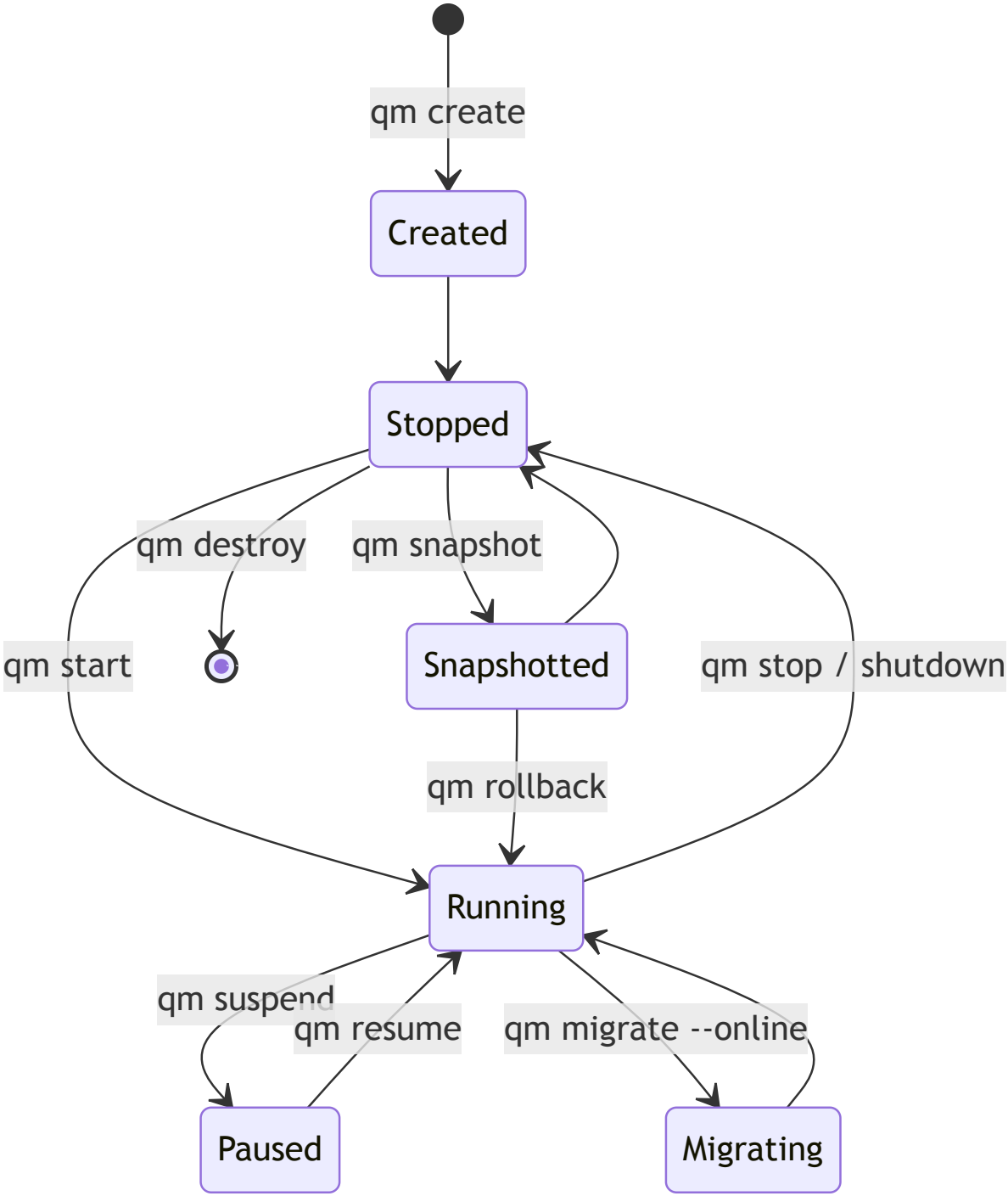

CPU with VT-x / AMD-V
KVM Kernel Module
QEMU Process - VM 100
QEMU Process - VM 101
Guest OS: Linux/Windows
VirtIO Block/Net/SCSI Drivers
Guest OS: Linux/Windows
VirtIO Block/Net/SCSI Drivers

Each running VM is literally **one QEMU process** on the host — you can see it with `ps aux | grep kvm` and inspect its live command line, which `qm showcmd <vmid>` will print for you.

VM Lifecycle

```
qm create 100 --name web01 --memory 4096 --cores 2 \
  --net0 virtio,bridge=vmbr0 --scsihw virtio-scsi-single \
  --scsi0 local-lvm:32 --ostype l26

qm start 100
qm status 100
qm shutdown 100
```


qm create
Created
Stopped
qm destroy
qm snapshot
qm start
Snapshotted
qm stop / shutdown
qm rollback
Running
qm suspend
qm resume
qm migrate --online
Paused
Migrating

3. Creating & Configuring Virtual Machines

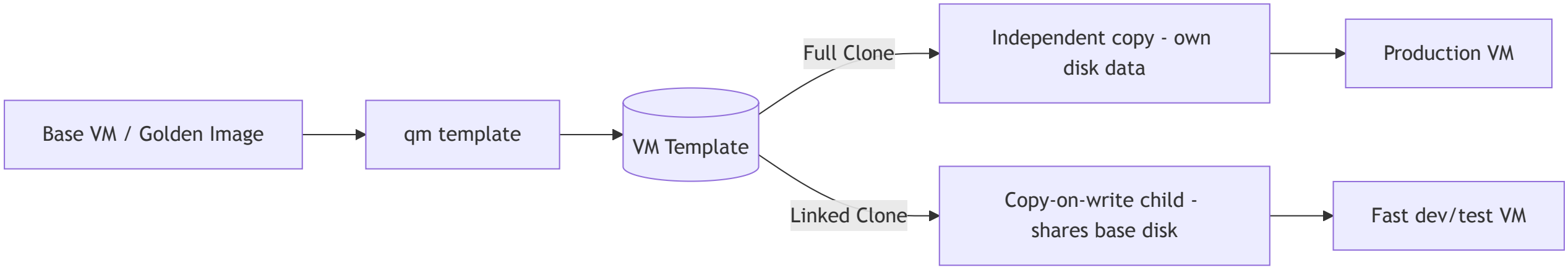
Key VM configuration areas you'll touch on nearly every guest:

Setting	Notes
Machine type	`q35` (modern, PCIe) vs `pc-i440fx` (legacy)
BIOS	SeaBIOS (legacy) vs OVMF (UEFI, required for Secure Boot)
CPU type	`host` (best perf, no live-migration across CPU vendors) vs a named model (`x86-64-v2-AES`) for portability
Disk bus	VirtIO SCSI (recommended), SATA, IDE (legacy guests only)
Display	VirtIO-GPU/SPICE for desktops, plain VGA/serial for headless servers

```
qm set 100 --cpu host --bios ovmf --machine q35
qm set 100 --efidisk0 local-lvm:1,efitype=4m,pre-enrolled-keys=1
```

4. VM Templates & Cloning

Converting a fully-configured VM into a **template** locks it (read-only) and allows fast cloning — either a **full clone** (independent copy) or a **linked clone** (copy-on-write, shares the base disk).


Base VM / Golden Image
qm template
VM Template
Full Clone
Linked Clone
Independent copy - own disk data
Copy-on-write child - shares base disk
Production VM
Fast dev/test VM

```
# Prepare a golden image, then template it
qm template 9000

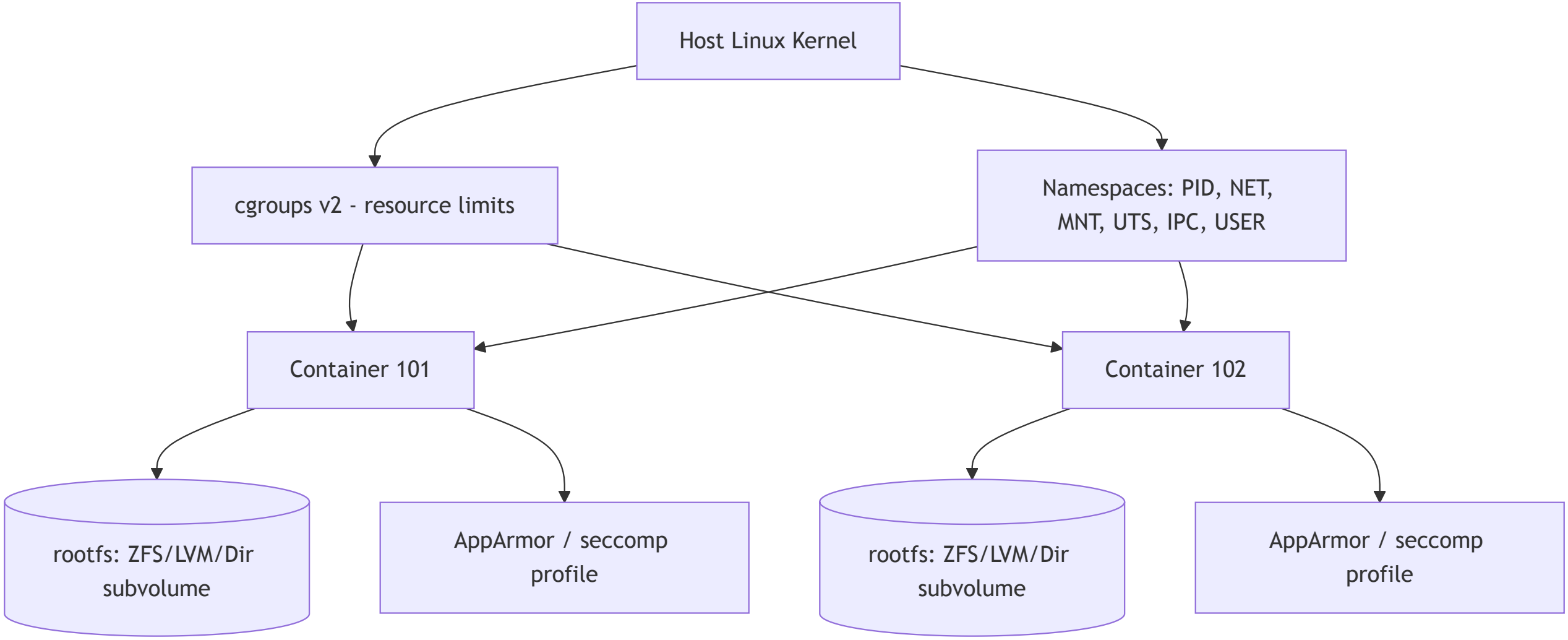
# Full clone — production, independent disk
qm clone 9000 101 --name web02 --full

# Linked clone — fast, disposable dev/test VM
qm clone 9000 102 --name web-test
```

Linked clones are much faster to create but **depend on the base template's disk remaining intact** — never delete a template that has linked clones.

5. LXC Containers — Concepts & Use Cases

Containers share the **host kernel** and rely on Linux **namespaces** (PID, NET, MNT, UTS, IPC, USER) for isolation and **cgroups v2** for resource limits — no hardware emulation involved.


Host Linux Kernel
cgroups v2 - resource limits
Namespaces: PID, NET, MNT, UTS, IPC, USER
Container 101
Container 102
rootfs: ZFS/LVM/Dir subvolume
AppArmor / seccomp profile
rootfs: ZFS/LVM/Dir subvolume
AppArmor / seccomp profile

```
pct create 300 local:vztmpl/debian-13-standard_13.6-1_amd64.tar.zst \
  --hostname app01 --memory 1024 --cores 1 \
  --net0 name=eth0,bridge=vmbr0,ip=dhcp \
  --unprivileged 1

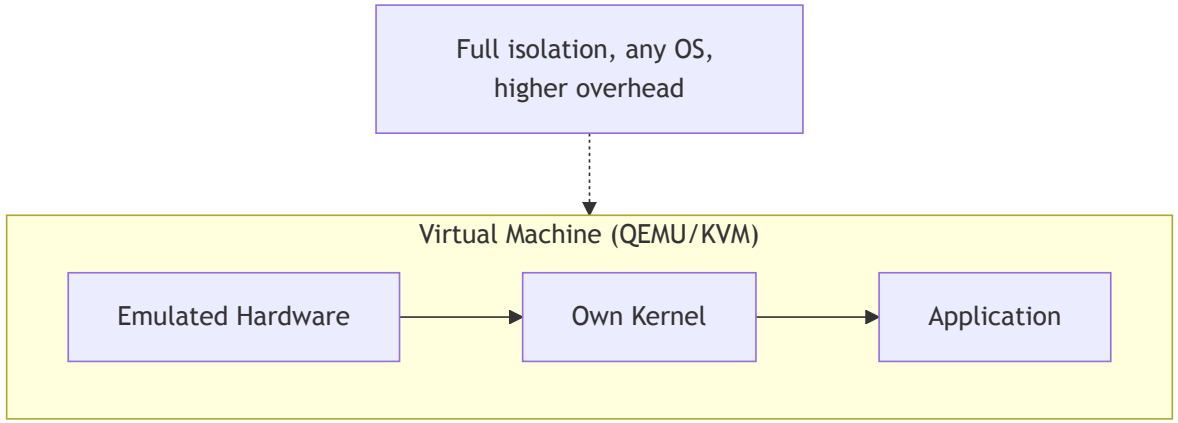
pct start 300
pct enter 300
```

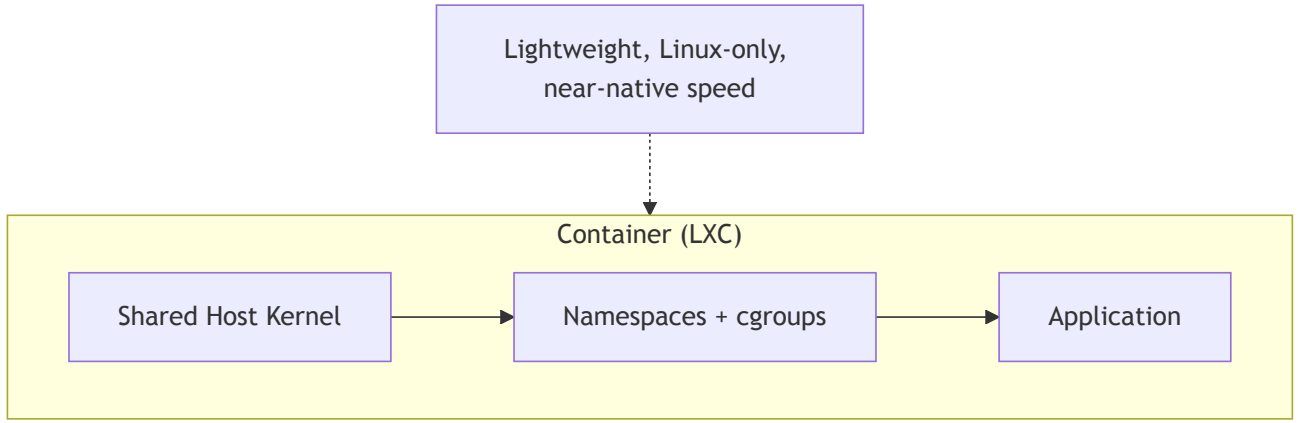
Containers are created **unprivileged by default** since Proxmox VE 9 — this maps container root to an unprivileged host UID range, significantly reducing the impact of a container breakout.

6. Containers vs. VMs — Choosing the Right Workload

Criteria	Choose a VM	Choose a Container
Guest OS	Windows, BSD, any non-Linux	Linux only
Isolation requirement	Strong / untrusted tenants	Trusted, same-org workloads
Density	Lower (full kernel per guest)	Higher (shared kernel)
Boot time	Seconds	Milliseconds
Kernel-level features needed	Custom kernel modules, nested virt	Rare — must be host-compatible

Rule of thumb: **default to LXC** for Linux services you trust and control; use a **VM** whenever you need OS diversity, strong isolation, or kernel-level customization.


Full isolation, any OS,
higher overhead
Virtual Machine (QEMU/KVM)
Emulated Hardware
Own Kernel
Application


Lightweight, Linux-only,
near-native speed
Container (LXC)
Shared Host Kernel
Namespaces + cgroups
Application

7. Resource Management — CPU, Memory, Disk

```
# CPU limits: sockets/cores + optional cpulimit/cpuunits
qm set 100 --sockets 1 --cores 4 --cpulimit 2 --cpuunits 2048

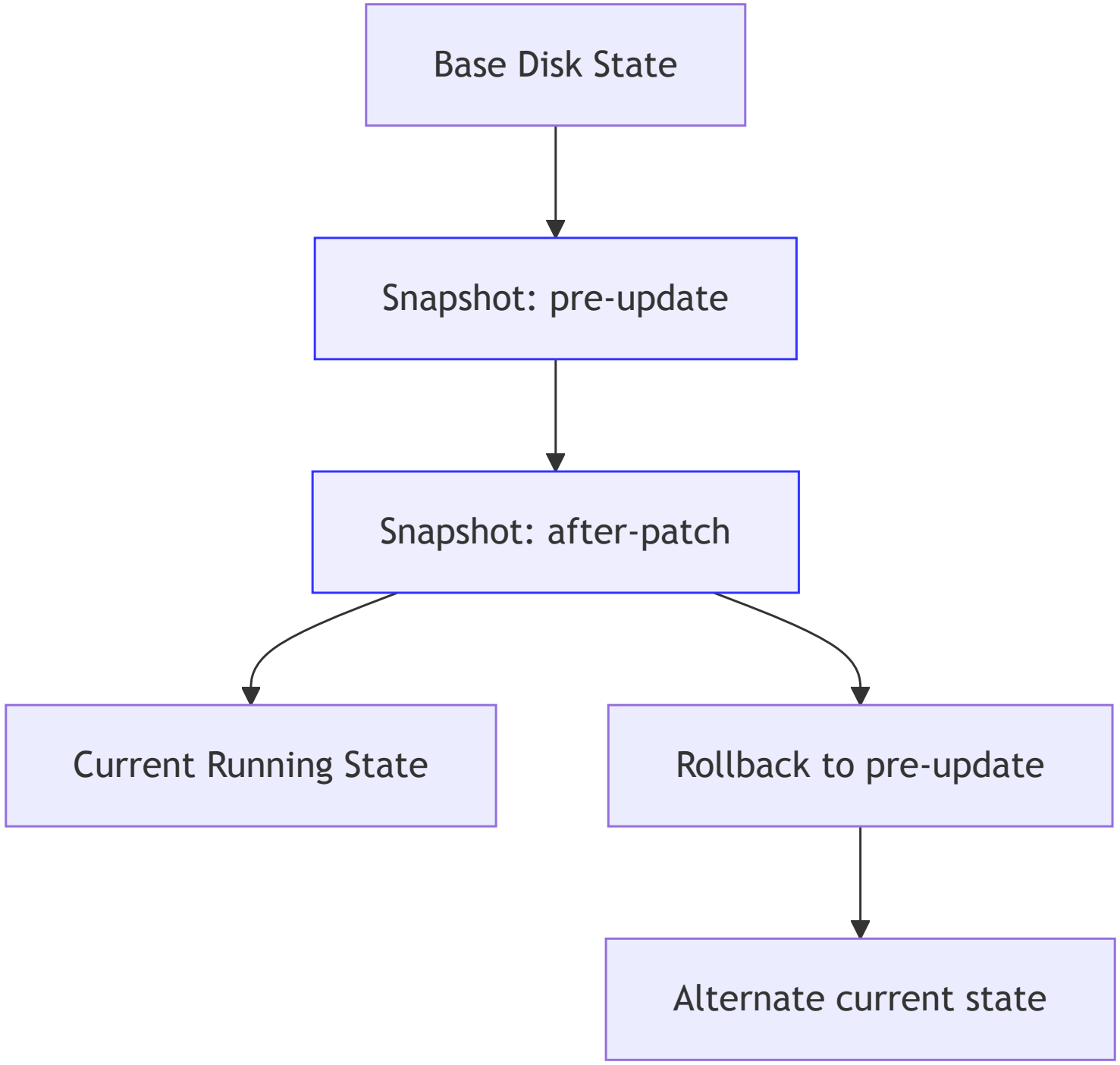
# Memory: fixed vs ballooning (min/max with balloon device)
qm set 100 --memory 8192 --balloon 4096

# Disk: resize live, add a second disk
qm resize 100 scsi0 +20G
qm set 100 --scsi1 local-lvm:50
```

Ballooning lets the host reclaim idle guest memory back to the pool without stopping the VM — useful for over-subscribed clusters, but disable it for latency-sensitive database workloads.

8. Snapshots & Live Migration

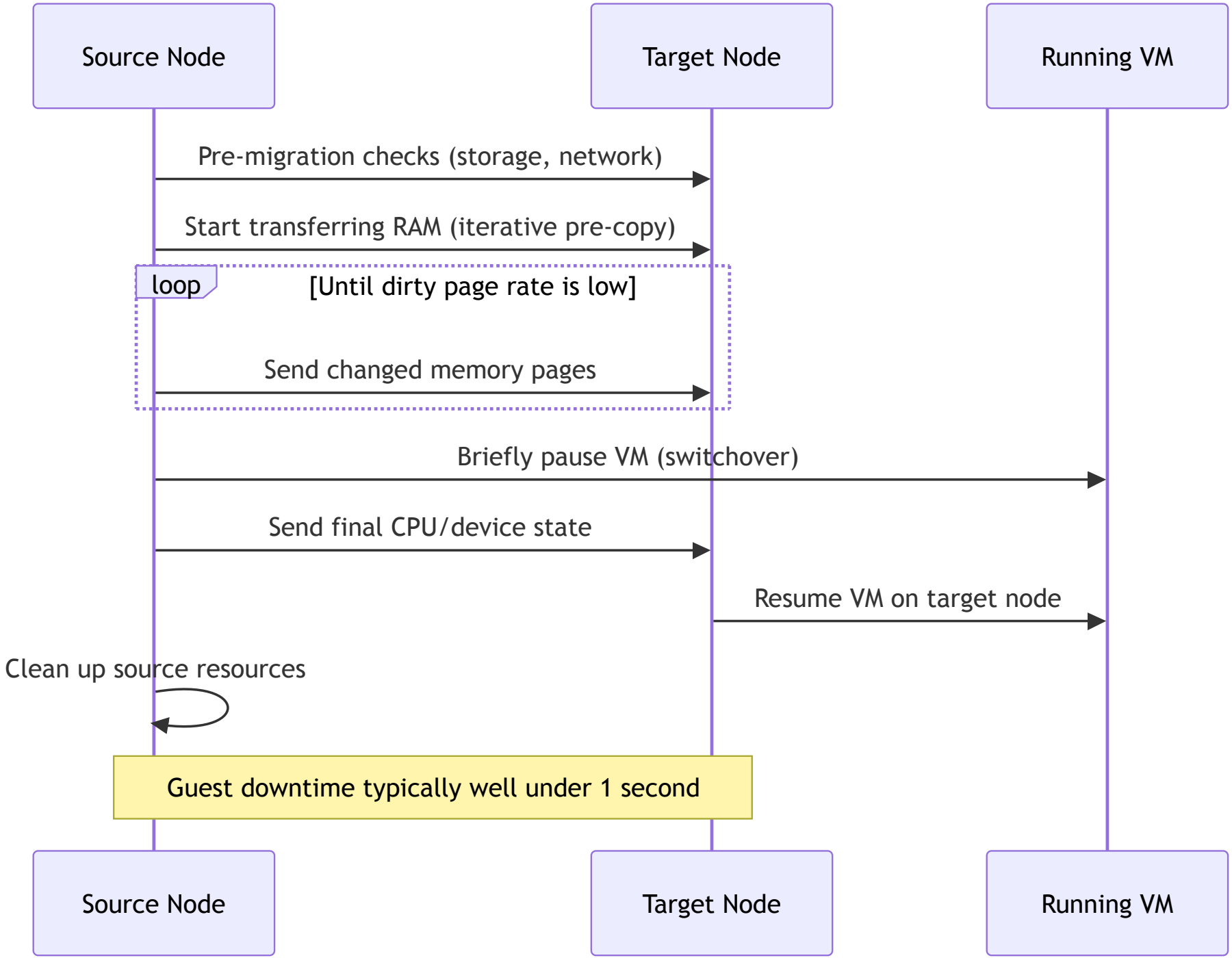
Snapshots capture disk state (and optionally RAM) at a point in time, letting you branch and roll back:


Base Disk State
Snapshot: pre-update
Snapshot: after-patch
Current Running State
Rollback to pre-update
Alternate current state

```
qm snapshot 100 pre-update --description "Before patching"
qm rollback 100 pre-update
qm delsnapshot 100 pre-update
```

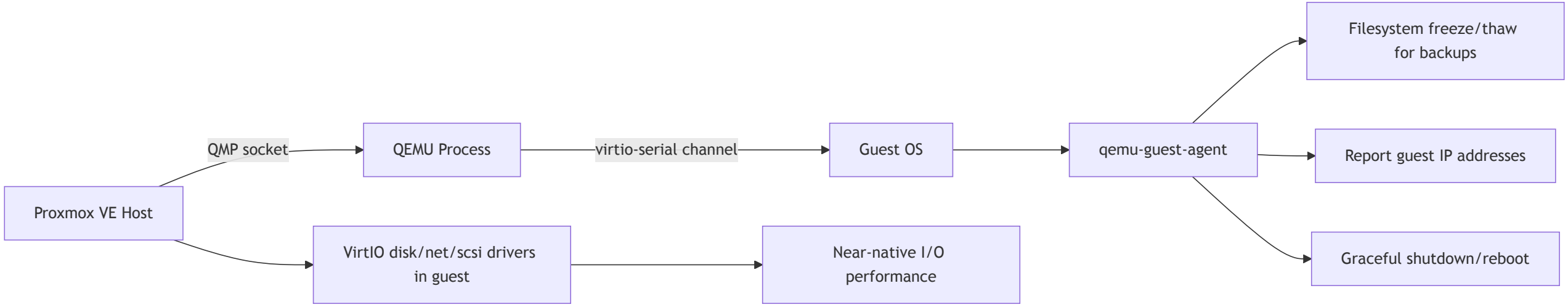
Live migration moves a *running* VM between nodes with sub-second downtime by iteratively copying memory pages before a brief final switchover:

```
qm migrate 100 pve2 --online
```


Source Node
Target Node
Running VM
Pre-migration checks (storage, network)
Start transferring RAM (iterative pre-copy)
loop
[Until dirty page rate is low]
Send changed memory pages
Briefly pause VM (switchover)
Send final CPU/device state
Resume VM on target node
Clean up source resources
Guest downtime typically well under 1 second
Source Node
Target Node
Running VM

9. VirtIO Drivers & the QEMU Guest Agent

VirtIO is a paravirtualized device interface — instead of emulating real hardware, the guest talks to QEMU through an efficient shared-memory protocol for disk, network, and SCSI.


Proxmox VE Host
QMP socket
QEMU Process
virtio-serial channel
Guest OS
qemu-guest-agent
Filesystem freeze/thaw for backups
Report guest IP addresses
Graceful shutdown/reboot
VirtIO disk/net/scsi drivers in guest
Near-native I/O performance

The **QEMU Guest Agent** (`qemu-guest-agent`, installed inside the guest) enables:

- Filesystem freeze/thaw for **crash-consistent backups**
- Reporting guest IP addresses back to the host
- Graceful shutdown/reboot instead of a hard power-off

```
qm set 100 --agent enabled=1,fstrim_cloned_disks=1
```

10. Hands-on Lab: Deploying VMs and Containers

In today's lab you will:

1. Create a VM from an ISO, install VirtIO drivers, enable the guest agent
2. Convert it into a template and create both a full and a linked clone
3. Deploy an unprivileged LXC container from a template
4. Take a snapshot, make a change, and roll back
5. (If a 2nd node is available) live-migrate a running VM

Day 2 — Key Takeaways

- Every VM is a **QEMU process** using **KVM** for hardware-accelerated CPU virtualization
- Containers share the host kernel via **namespaces + cgroups** — faster, Linux-only
- Templates + linked clones make **rapid, disk-efficient provisioning** possible
- Live migration and snapshots are built on the same underlying block/memory primitives
- **VirtIO + qemu-guest-agent** are essential for performance and backup consistency